

Alex First Blood

Manuel d'apprentissage

Les fondamentaux de l'informatique, en mots simples

Préparé pour Alexandre

Avant l'entrée en Bachelor 1 Cybersécurité

10 chapitres pour partir de zéro et arriver prêt.

Bienvenue, Alexandre

Respire un grand coup : tu n'as **rien** besoin de savoir pour lire ce manuel. On part vraiment de zéro. Chaque mot compliqué sera expliqué avec des exemples de la vie de tous les jours.

Ce manuel est ton **livre de cours**. Il explique le *pourquoi* et le *comment* des choses, lentement et simplement. À côté, tu as un **cahier d'exercices** (un fichier par semaine) avec les manipulations à faire sur ton ordinateur. Les deux vont ensemble : tu lis un chapitre ici, puis tu pratiques dans le cahier.

First Blood, c'est quoi (et pourquoi pas CyberHero) ?

Tu as peut-être déjà commencé « **Alex le CyberHero** » : c'était l'entraînement **offensif** — apprendre à attaquer pour comprendre comment les méchants pensent. C'est passionnant, mais c'est une **spécialisation**.

« **First Blood** », c'est autre chose : c'est la **base**. Ta première année de Bachelor ne parle pas que de hacking. Elle parle surtout de :

- **Développement** : écrire des programmes (en Python, en C, pour le web, en PHP).
- **Bases de données** : ranger et retrouver des informations.
- **Réseaux** : comment les ordinateurs se parlent.
- **Systèmes** : installer et administrer des serveurs.

La cybersécurité repose sur **tout ça**. On ne protège bien que ce qu'on comprend. Un expert en sécurité qui ne sait pas coder ni comment marche un réseau, c'est comme un serrurier qui n'a jamais vu de porte. First Blood te donne ces fondations, pour que tu arrives en cours **déjà à l'aise**.

Comment lire ce manuel

- **Ne saute pas les analogies.** Quand je compare Git à une machine à remonter le temps, ce n'est pas pour faire joli : c'est pour que l'idée reste collée dans ta tête.
- **Lis à voix haute les phrases que tu ne comprends pas.** Souvent, ça suffit à débloquer.
- **Tu as le droit de ne pas tout retenir du premier coup.** Le code, c'est comme un instrument de musique : on progresse en jouant, pas en lisant la partition dix fois.
- Les encadrés comme celui ci-dessous sont des **points à retenir absolument**.

À retenir : ce manuel se lit dans l'ordre. Chaque chapitre suppose que tu as compris le précédent. Prends ton temps.

La règle la plus importante

On va te donner des outils puissants. Avec eux, on peut faire de très bonnes choses... et de très mauvaises. Retiens ceci, c'est non négociable :

À retenir : tu codes, tu administres et tu expérimentes sur TES propres machines, ou sur des plateformes faites pour ça. Tu n'entres JAMAIS dans un ordinateur, un site ou un réseau qui ne t'appartient pas sans autorisation écrite. En France, c'est un délit (article 323-1 du Code pénal), même si tu ne casses rien. Ton savoir sert à construire et à protéger.

Le plan des 10 semaines

Voici le chemin, étape par étape. Chaque chapitre de ce manuel correspond à une semaine.

- **Semaine 1 — Linux, terminal & Git.** Le système des serveurs, la ligne de commande, et la machine à sauvegarder ton travail.
- **Semaine 2 — Python, les bases.** Ton premier vrai langage de programmation : variables, choix, répétitions, fonctions.
- **Semaine 3 — Python, les données.** Ranger l'information (listes, dictionnaires), lire des fichiers, et un mini-projet complet.
- **Semaine 4 — Le langage C.** Plonger sous le capot : la mémoire, les pointeurs, ce qui se passe vraiment dans la machine.
- **Semaine 5 — Le web côté visible.** Construire des pages (HTML), les habiller (CSS), les animer (JavaScript).
- **Semaine 6 — Le web côté caché.** PHP : le code qui tourne sur le serveur et fabrique les pages.
- **Semaine 7 — Les bases de données.** Stocker et interroger des montagnes d'informations avec SQL, et les dessiner avant de les construire (UML).
- **Semaine 8 — Le réseau.** Comment les machines se parlent : adresses, chemins, sous-réseaux.
- **Semaine 9 — Systèmes & virtualisation.** Monter des serveurs et des « ordinateurs dans l'ordinateur ».
- **Semaine 10 — Le grand final.** Un peu de cryptographie, le cadre légal (RGPD), et un projet qui réunit tout.

Deux fils rouges parcourent toutes les semaines : un **coin maths** (juste ce qu'il faut, au bon moment) et un **coin anglais** (le vocabulaire du métier).

Allez, on y va. Tourne la page.

Chapitre 1 — Linux, Git et toi

Pourquoi on commence par Linux

Imagine que tu apprends la cuisine. Avant les recettes, tu dois connaître ta **cuisine** : où sont les couteaux, comment marche le four, où on range les ingrédients. En informatique, cette cuisine, c'est le **système d'exploitation**.

Tu connais sûrement Windows ou macOS. Il en existe un troisième, gratuit et ouvert : **Linux**. Et c'est lui le roi du monde professionnel : **la grande majorité des serveurs** qui font tourner les sites web, les applications et les services en ligne sont sous Linux. Apprendre Linux, ce n'est pas une option : c'est le terrain sur lequel tu vas jouer toute ta carrière.

À retenir : Windows pour le grand public, Linux pour les serveurs et les pros. En cybersécurité et en développement, tu travailleras énormément sous Linux.

Tu n'as pas besoin de jeter ton ordinateur. Tu peux avoir Linux **à l'intérieur** de Windows (avec WSL), ou dans une **machine virtuelle** (un ordinateur simulé dans ton ordinateur). Le cahier d'exercices te guide pour l'installer.

Le terminal : parler à la machine avec des mots

Sous Windows, tu cliques sur des icônes. Sous Linux, tu peux faire pareil, mais les pros utilisent surtout le **terminal** : une fenêtre où tu **tapes des commandes**.

Pourquoi taper, alors qu'on pourrait cliquer ? Parce que c'est **plus rapide, plus précis**, et surtout **automatisable**. Quand tu sais écrire une commande, tu peux la répéter mille fois sans effort. Un clic, lui, ne se répète pas tout seul.

Pense au terminal comme à **parler à un assistant très obéissant mais très bête** : il fait *exactement* ce que tu dis, ni plus ni moins. Si tu lui dis « supprime tout », il supprime tout, sans te demander si tu es sûr. D'où la prudence.

À retenir : le terminal exécute tes ordres à la lettre. C'est sa force (puissance, automatisation) et son danger (une faute de frappe peut faire mal). On réfléchit avant d'appuyer sur Entrée.

Quelques ordres de base, en français :

- **pwd** : « où suis-je ? »
- **ls** : « montre-moi ce qu'il y a ici »
- **cd** : « emmène-moi dans ce dossier »
- **mkdir** : « crée un dossier »
- **cat** : « affiche le contenu de ce fichier »

Les permissions : qui a le droit de quoi

Sous Linux, chaque fichier a un **propriétaire** et des **droits**. C'est comme les clés d'un immeuble : certaines personnes peuvent entrer dans certaines pièces, d'autres non.

Trois actions possibles sur un fichier : le **lire**, l'**écrire** (le modifier), et l'**exécuter** (le lancer, s'il s'agit d'un programme). Et trois publics : **toi** (le propriétaire), **ton groupe**, et **tout le monde**.

Tu verras souvent des nombres comme 755 ou 644. C'est juste une façon courte d'écrire ces droits. Pas de panique : on les manipule tellement souvent qu'ils finiront par te sembler naturels. Le cahier d'exercices t'apprend à les lire.

Un script : enchaîner les ordres

Quand tu tapes plusieurs commandes souvent, tu peux les écrire dans un **fichier** et les lancer toutes d'un coup. Ce fichier s'appelle un **script**. C'est ta première forme de programmation : tu ne cliques plus, tu **automatises**.

Ton tout premier script affichera juste un message de bienvenue avec la date. Rien de spectaculaire — mais c'est le moment où tu passes de « celui qui utilise l'ordinateur » à « celui qui lui donne des ordres ».

Git : la machine à remonter le temps

Voici une situation que tout débutant connaît. Tu travailles sur un projet. Ça marche. Tu modifies quelque chose. Ça casse. Et là... impossible de te souvenir de ce que tu as changé. Tu as bien gardé `projet_final.zip`, puis `projet_final_v2.zip`, puis `projet_vraiment_final.zip`... mais lequel marchait, déjà ?

Git résout ça pour de bon. C'est un outil qui prend une **photo** de ton projet à chaque étape importante. Chaque photo s'appelle un **commit**. Tu peux à tout moment :

- revenir à une photo précédente (annuler une bêtise),
- voir exactement ce qui a changé entre deux photos,
- travailler à plusieurs sans vous écraser.

À retenir : un commit, c'est une photo de ton projet à un instant donné, avec une étiquette qui dit ce que tu as fait. Git garde toutes les photos dans l'ordre : c'est ton album, ta machine à remonter le temps.

Et **GitHub** dans tout ça ? C'est un **site web** où tu ranges tes albums Git **en ligne**. Deux avantages énormes :

1. **Sauvegarde** : même si ton ordinateur brûle, ton code est en sécurité.
2. **Portfolio** : un recruteur peut voir ce que tu as construit. En 2026, un bon profil GitHub vaut parfois plus qu'un CV.

Attention à ne pas confondre deux gestes :

- **commiter** = prendre une photo, **sur ton ordinateur** (ça marche même sans Internet) ;
- **pousser** (*push*) = envoyer tes photos **sur GitHub**.

Le cycle que tu répéteras toute ta vie de dev : je modifie `git add` (je prépare la photo) `git commit` (clic !) `git push` (je publie).

Ce qu'il faut retenir de ce chapitre

- Linux est le système des serveurs : tu vas l'utiliser partout.
- Le terminal te rend puissant : tu donnes des ordres précis et répétables.
- Les permissions disent qui a le droit de lire, écrire, exécuter.
- Un script automatise une suite de commandes.
- Git photographie ton projet étape par étape ; GitHub le sauvegarde en ligne et le montre au monde.

Maintenant, file dans le cahier de la **semaine 1** et mets les mains dans le cambouis. On ne devient pas un pro en lisant — on le devient en tapant des commandes.

Chapitre 2 — Python, ton premier langage

Programmer, c'est écrire des recettes

Tu sais déjà donner des ordres à la machine (le terminal, chapitre 1). Programmer, c'est aller plus loin : écrire une **suite d'instructions** que l'ordinateur exécutera tout seul. On appelle ça un **algorithme** — un mot impressionnant pour une idée simple : *une recette de cuisine*.

Faire des pâtes : *faire bouillir l'eau saler plonger les pâtes attendre 10 minutes égoutter*. C'est précis, ordonné, sans ambiguïté. Un programme, c'est pareil, mais écrit dans un langage que la machine comprend.

On commence par **Python**, et ce n'est pas un hasard : il est **lisible** (ça ressemble presque à de l'anglais), **pardonnant** pour les débutants, et c'est **le langage enseigné en première année**. Ce que tu apprends ici, tu le retrouveras en cours dès les premières semaines.

À retenir : un algorithme est une suite d'étapes précises pour résoudre un problème. Un langage de programmation (comme Python) sert à écrire cet algorithme pour que la machine l'exécute.

Les variables : des boîtes étiquetées

Une **variable**, c'est une boîte avec une étiquette, dans laquelle tu ranges une valeur. Tu écris `age = 18`, et désormais le mot `age` contient `18`. Tu peux le réutiliser, le modifier, le combiner.

Les valeurs ont un **type** : un nombre entier (`int`), un nombre à virgule (`float`), du texte (`str`), ou un vrai/faux (`bool`). Le type, c'est la **nature** de ce qu'il y a dans la boîte — et ça compte, car on n'additionne pas du texte comme des nombres.

Faire des choix : les conditions

Un programme intéressant **réagit**. « Si la note est au-dessus de 10, c'est validé, sinon c'est à refaire. » En Python : `if`, `elif` (sinon si), `else` (sinon).

Un détail crucial de Python : l'**indentation**. Les espaces en début de ligne ne sont pas décoratifs — ils disent ce qui est « à l'intérieur » du `if`. Décaler le texte, c'est dire à Python « ceci appartient au bloc du dessus ». C'est déroutant au début, puis ça devient une seconde nature (et ça force à écrire du code bien rangé).

À retenir : avec if/elif/else, ton programme prend des décisions. En Python, l'indentation (les espaces à gauche) délimite les blocs : ce n'est pas optionnel.

Répéter : les boucles

L'ordinateur ne se fatigue jamais : il adore répéter. C'est le rôle des **boucles**. La boucle **for** répète un nombre **connu** de fois (« pour chaque élément de cette liste... »). La boucle **while** répète **tant qu'**une condition reste vraie (« tant que le joueur n'a pas deviné... »).

Attention au piège classique : si la condition d'un **while** ne devient **jamais** fausse, la boucle tourne à l'infini et le programme se fige. Vérifie toujours qu'il y a une porte de sortie.

Ranger le code : les fonctions

Quand tu fais souvent la même chose, tu ne réécris pas le code à chaque fois : tu en fais une **fonction**, un bloc nommé que tu peux appeler à volonté. `def aire(l, h):`
`return l * h` définit une fonction « aire » ; ensuite `aire(4, 3)` te donne `12`, autant de fois que tu veux.

C'est la règle d'or du code propre : **si tu copies-colles trois fois le même bout de code, c'est qu'il te faut une fonction**. Un programme bien fait, c'est un assemblage de petites fonctions claires, chacune faisant une seule chose.

Ce qu'il faut retenir de ce chapitre

- Un programme est une recette : une suite d'instructions précises.
- Les variables rangent des valeurs ; chaque valeur a un type (`int`, `float`, `str`, `bool`)
- `if/elif/else` font des choix ; l'indentation délimite les blocs.
- `for` et `while` répètent ; gare à la boucle infinie.
- Les fonctions (`def`) rangent et réutilisent le code.

Direction le cahier de la **semaine 2** : tu vas coder ton premier vrai jeu, le « nombre mystère ». Le moment où programmer devient amusant.

Chapitre 3 — Ranger l'information

Une variable ne suffit plus

Au chapitre 2, une variable contenait une seule valeur : un âge, un prénom. Mais comment ranger **la liste des courses, un carnet de contacts, les notes d'une classe** ? Tu ne vas pas créer `note1`, `note2`, `note3`... jusqu'à `note30`. Il te faut des **structures de données** : des boîtes capables de contenir **plusieurs choses**, organisées.

La liste : une suite ordonnée

La **liste** est la structure la plus courante : une suite de valeurs, dans l'ordre, dans une seule variable. `courses = ["pain", "lait", "œufs"]`. Tu accèdes au premier élément avec `courses[0]` (en informatique, **on compte à partir de zéro** — un réflexe à prendre). Tu peux ajouter (`.append`), enlever, trier, compter (`len`), parcourir avec une boucle `for`.

À retenir : une liste range plusieurs valeurs ordonnées. On accède aux éléments par leur position (l'indice), en commençant à 0.

Le dictionnaire : associer une clé à une valeur

Parfois, l'ordre n'a pas d'importance : ce qu'on veut, c'est **retrouver une information par son nom**. C'est le rôle du **dictionnaire**. Comme un vrai dictionnaire associe un mot à sa définition, le dictionnaire Python associe une **clé** à une **valeur** : `eleve = {"nom": "Alex", "age": 18}`. Tu retrouves l'âge avec `eleve["age"]`.

C'est puissant : retrouver une valeur par sa clé est **quasi instantané**, même dans un dictionnaire géant. Là où parcourir une liste d'un million d'éléments prendrait du temps, le dictionnaire répond du tac au tac.

Il existe deux cousins : le **tuple** (une liste qu'on ne peut plus modifier, pour des données fixes) et l'**ensemble** (set, une collection sans doublons).

À retenir : la liste se parcourt par position ; le dictionnaire se consulte par clé. Choisir la bonne structure selon l'usage, c'est déjà bien programmer.

Les fichiers : se souvenir après l'extinction

Jusqu'ici, dès que ton programme s'arrête, tout est oublié. Pour qu'il **se souvienne** d'une fois sur l'autre, il faut écrire sur le **disque** : dans un **fichier**.

On ouvre un fichier en précisant ce qu'on veut faire : le **lire**, y **écrire** (en écrasant), ou **ajouter** à la fin. La bonne pratique est d'utiliser le bloc `with open(...)`, qui referme automatiquement le fichier quand on a fini — comme on referme le robinet

après s'être servi.

Un format très courant pour stocker des données structurées est le **JSON** : du texte lisible qui ressemble exactement aux dictionnaires et listes Python. Python sait le lire et l'écrire en une ligne.

Quand ça se passe mal : gérer les erreurs

Un utilisateur tape « dix-huit » au lieu de « 18 » ? Le fichier n'existe pas ? Au lieu de planter, un bon programme **anticipe**. Le bloc `try / except` te permet de dire : « essaie ceci ; si ça rate, fais cela plutôt ». C'est la différence entre un programme qui crash brutalement et un programme robuste qui garde le contrôle.

Ce qu'il faut retenir de ce chapitre

- Les structures de données rangent **plusieurs** valeurs : liste (ordonnée), dictionnaire (par clé), tuple (fixe), set (sans doublons).
- On accède à une liste par position, à un dictionnaire par clé.
- Les fichiers permettent de **se souvenir** d'une exécution à l'autre ; le JSON est un format pratique.
- `try/except` rend un programme robuste face aux imprévus.

Au cahier de la **semaine 3**, tu construis ton premier programme qui **se souvient** : un gestionnaire de tâches qui sauvegarde tout dans un fichier. Ta première vraie petite application.

Chapitre 4 — Sous le capot : le C

Pourquoi quitter le confort de Python ?

Python est merveilleux : il te cache tout ce qui est compliqué. Mais justement — pour devenir un pro, il faut **soulever le capot** et voir comment la machine fonctionne vraiment. C'est le rôle du langage **C**.

Le C, c'est le grand-père des langages modernes. Linux est écrit en C. Windows, en grande partie. Python lui-même est écrit en C ! Apprendre le C, ce n'est pas apprendre un langage de plus : c'est **comprendre l'informatique de l'intérieur** — la mémoire, la vitesse, ce qui se passe vraiment quand un programme tourne.

À retenir : Python te protège, le C t'expose. C'est plus exigeant, mais c'est en C qu'on comprend comment marche réellement un ordinateur.

Compilé vs interprété

Voici une différence fondamentale. Python est **interprété** : un programme (l'interpréteur) lit ton code ligne par ligne et l'exécute à la volée. Le C est **compilé** : avant de pouvoir lancer ton programme, tu dois le **traduire** entièrement en langage machine (du binaire) avec un outil appelé **compilateur** (sous Linux, c'est **gcc**).

L'analogie : Python, c'est un interprète qui traduit une conversation en direct, phrase par phrase. Le C, c'est faire traduire tout un livre à l'avance — c'est plus de travail au départ, mais ensuite la lecture est ultra-rapide. C'est pour ça que les programmes en C sont **beaucoup plus rapides**.

Tout est plus précis (et plus strict)

En C, tu dois **tout déclarer** : le type de chaque variable, et même la taille qu'elle occupe en mémoire. Un **int** prend un nombre fixe d'octets, un **char** (un caractère) en prend un seul. Rien n'est automatique. Le compilateur est sévère : la moindre erreur de type ou de point-virgule oublié, et il refuse de compiler. Ça paraît pénible... mais c'est une **excellente école de rigueur**.

Le grand morceau : les pointeurs

Voici LE concept que tout le monde redoute, et qui fait toute la beauté du C. Pour le comprendre, une image : la mémoire de l'ordinateur est une **immense rue de petites maisons**, chacune avec une **adresse**. Quand tu crées une variable, elle est rangée dans une de ces maisons.

Un **pointeur**, c'est une variable spéciale qui ne contient pas une valeur... mais l'**adresse** d'une autre variable. Au lieu de contenir « 18 », il contient « la valeur 18 se trouve à la maison numéro 7482 ».

Deux symboles vont avec :

- `&age` veut dire « l'**adresse** de la variable `age` » (où elle habite).
- `*p` veut dire « la **valeur qui se trouve** à l'adresse contenue dans `p` » (ce qu'il y a dans la maison).

Pourquoi c'est génial ? Parce qu'avec un pointeur, une fonction peut **modifier directement** une variable qui vit ailleurs, partager de gros volumes de données sans les recopier, et construire des structures complexes. C'est aussi la source des bugs les plus célèbres (et de failles de sécurité entières). Le maîtriser te place très au-dessus du débutant moyen.

À retenir : un pointeur ne contient pas une valeur, mais l'adresse où trouver la valeur. `&` donne l'adresse, donne le contenu à cette adresse. Dessine des cases mémoire sur papier — c'est comme ça que ça « fait tilt ».

Ce qu'il faut retenir de ce chapitre

- Le C expose la machine : mémoire, types de taille fixe, rigueur.
- Compilé (C, traduit avant par `gcc`) ≠ interprété (Python, lu à la volée).
- Un pointeur stocke une **adresse** ; `&` = adresse de, `*` = contenu à cette adresse.
- C'est le concept le plus difficile et le plus formateur de ta première année.

Au cahier de la **semaine 4**, tu écriras le programme qui échange deux variables grâce aux pointeurs — le petit exercice qui fait enfin « tilt ».

Chapitre 5 — Le web côté visible

Ce qui se passe quand tu ouvres une page

Tu tapes une adresse, et une page s'affiche : du texte, des images, des couleurs, des boutons. Mais qui fabrique tout ça ? En réalité, ton navigateur reçoit du **code** et le transforme en page jolie. Ce code, côté visible, repose sur **trois langages** qui travaillent ensemble.

L'analogie de la maison résume tout :

- **HTML** = les **murs et les pièces** (la structure, le contenu).
- **CSS** = la **peinture et la décoration** (l'apparence).
- **JavaScript** = l'**électricité et les interrupteurs** (ce qui réagit quand on agit).

À retenir : HTML structure, CSS habille, JavaScript anime. Les trois ensemble font une page web vivante. Et le plus beau : aucun logiciel compliqué, juste un éditeur de texte et un navigateur.

HTML : la charpente

Le **HTML** décrit la structure avec des **balises** entre chevrons : `<h1>` pour un grand titre, `<p>` pour un paragraphe, `<a>` pour un lien, `<img>` pour une image, `<ul>` et `<li>` pour une liste. La plupart vont par paires : une balise ouvrante et une fermante, qui encadrent le contenu.

Le HTML ne s'occupe **que du sens** : « ceci est un titre », « ceci est un paragraphe », « ceci est un lien vers cette adresse ». Pas des couleurs, pas de la disposition — ça, c'est le travail du CSS.

CSS : l'habillage

Le **CSS** vient styliser le HTML. Tu écris des **règles** : « les titres sont rouges et centrés », « le fond est sombre », « ce bouton a des coins arrondis ». Tu choisis **quels** éléments styliser avec des *sélecteurs* (par type de balise, par classe, par identifiant).

Deux notions reviennent partout : le **modèle de boîte** (chaque élément est une boîte avec une marge, une bordure, un espace intérieur et un contenu) et **Flexbox** (un système moderne pour disposer les éléments proprement, les aligner, les répartir). Avec le CSS, la même page HTML peut passer d'austère à magnifique.

JavaScript : la vie

Le HTML et le CSS donnent une page **figée**. Le **JavaScript** la rend **interactive** : un bouton qui réagit au clic, un menu qui s'ouvre, un compteur qui s'incrémente, un formulaire qui se vérifie sans recharger la page. JavaScript s'exécute **dans le**

navigateur et peut modifier la page en direct — on dit qu'il manipule le **DOM** (la représentation de la page que le navigateur garde en mémoire).

Le schéma mental : tu **attrapes** un élément de la page (`document.getElementById`), puis tu lui dis de **réagir** à un événement (`addEventListener("click", ...)`), et dans la réaction, tu **modifies** la page.

À retenir : sans JavaScript, une page est une affiche. Avec, elle devient une application. C'est lui qui réagit aux actions de l'utilisateur.

Ce qu'il faut retenir de ce chapitre

- Une page web visible repose sur trois langages : HTML (structure), CSS (style), JavaScript (interactivité).
- HTML = des balises qui décrivent le sens du contenu.
- CSS = des règles de style appliquées via des sélecteurs ; pense « modèle de boîte » et Flexbox.
- JavaScript anime la page en réagissant aux actions de l'utilisateur (le DOM).

Au cahier de la **semaine 5**, tu construis ta carte de visite numérique interactive — et tu la publies en ligne gratuitement. Ta première vitrine sur le web.

Chapitre 6 — Le web côté caché

Le problème du front tout seul

Au chapitre 5, tout se passait dans le navigateur. Mais imagine un site qui affiche **ton compte bancaire**. Les données ne sont pas dans la page (sinon n'importe qui les verrait dans le code !) : elles arrivent d'un **serveur**, un ordinateur distant qui vérifie qui tu es, va chercher tes infos, et te renvoie une page sur mesure. Ce travail invisible, c'est le **back-end**.

Le langage qu'on apprend pour ça est **PHP**. Il est partout : WordPress, Wikipédia, une grande part du web tourne grâce à lui. Le principe : **PHP fabrique le HTML sur le serveur, puis l'envoie au navigateur**. L'utilisateur reçoit une page finie ; il ne voit jamais ton code PHP. C'est ça, la grande différence avec JavaScript.

À retenir : le front (HTML/CSS/JS) s'exécute chez l'utilisateur ; le back (PHP) s'exécute sur le serveur. Le PHP construit la page avant de l'envoyer, et garde le code caché.

PHP ressemble à ce que tu connais déjà

Bonne nouvelle : après Python et le C, le PHP te paraîtra familier. On y retrouve les variables, les conditions **if**, les boucles **for** et **while**, les fonctions, les tableaux. Trois habitudes à prendre : **toute variable commence par `\$`** (**\$prenom**), chaque instruction finit par **;**, et les blocs sont délimités par des accolades **{ }** (pas par l'indentation comme Python).

Le code PHP s'insère dans le HTML entre les balises `<?php ... ?>`, ce qui permet de mélanger logique et affichage.

Le cœur du métier : les formulaires

À quoi sert vraiment le back-end ? À **traiter ce que l'utilisateur envoie**. Un formulaire (champ de saisie, bouton « envoyer ») transmet ses données au serveur, qui les récupère et agit : enregistrer une inscription, vérifier un mot de passe, poster un message.

Et ici, une leçon capitale, valable pour toute ta carrière : **ne fais JAMAIS confiance à ce que l'utilisateur envoie**. Un visiteur malveillant peut envoyer n'importe quoi pour piéger ton site (c'est la base des attaques web que tu as vues dans CyberHero). On **valide** et on **nettoie** toujours les données reçues avant de les utiliser ou de les afficher.

À retenir : recevoir et traiter les données d'un formulaire, c'est le cœur du back-end. Et la règle d'or : toute donnée venant de l'utilisateur est suspecte tant qu'on ne l'a pas vérifiée.

Se souvenir de l'utilisateur : les sessions

Le web a une particularité étrange : il **oublie**. Chaque page est, par défaut, indépendante de la précédente. Alors comment un site sait-il que tu es resté connecté en passant d'une page à l'autre ? Grâce aux **sessions** : un petit espace mémoire côté serveur qui retient « cet utilisateur est connecté en tant qu'Alex ». C'est le mécanisme derrière **tout système de connexion**. On le combinera, en semaine 7, avec une base de données pour stocker les vrais comptes.

Ce qu'il faut retenir de ce chapitre

- Le back-end (PHP) tourne sur le serveur et fabrique les pages que le navigateur reçoit.
- PHP ressemble aux langages déjà vus : **\$** devant les variables, **;**, accolades.
- Traiter les formulaires est son rôle central ; on ne fait jamais confiance aux données reçues.
- Les sessions permettent de « se souvenir » d'un utilisateur d'une page à l'autre (base de la connexion).

Au cahier de la **semaine 6**, tu montes une page de connexion avec sessions. Tu touches au cœur de toute application web sérieuse.

Chapitre 7 — Ranger des montagnes de données

Le fichier ne suffit plus

En semaine 3, tu sauvegardais tes tâches dans un fichier. Parfait pour dix tâches. Mais imagine un site avec **un million de clients**, des recherches instantanées, des dizaines d'utilisateurs qui écrivent **en même temps**, et l'interdiction absolue de perdre la moindre donnée. Un simple fichier s'effondre. Il faut un outil conçu pour ça : une **base de données**, pilotée par un **SGBD** (système de gestion de base de données) comme MySQL/MariaDB.

Le modèle relationnel : des tableaux reliés

La forme la plus répandue est la base **relationnelle**. L'idée est simple et géniale : on range les données dans des **tables**, comme des feuilles de tableur. Une table **clients**, une table **commandes**, une table **produits**... Chaque **ligne** est un enregistrement (un client), chaque **colonne** un champ (son nom, son email).

Une colonne spéciale, la **clé primaire** (souvent un **id**), donne à chaque ligne un identifiant **unique** — comme un numéro de sécurité sociale. Impossible de confondre deux clients, même s'ils ont le même nom.

À retenir : une base relationnelle range les données en tables (lignes = enregistrements, colonnes = champs). La clé primaire identifie chaque ligne de façon unique.

SQL : parler à la base

Pour dialoguer avec la base, un langage universel : le **SQL**. Il est étonnamment proche de l'anglais courant. Quatre actions de base, qu'on résume par « CRUD » :

- **lire** : `SELECT ... FROM ...` (« sélectionne ces colonnes dans cette table ») ;
- **créer** : `INSERT INTO ...` (« insère une ligne ») ;
- **modifier** : `UPDATE ... SET ...` ;
- **supprimer** : `DELETE FROM ...`.

Et un mot magique : **WHERE**, qui **filtre** (« seulement les clients de Paris »). Attention, leçon qui sauve des carrières : un **UPDATE** ou un **DELETE** **sans** **WHERE** s'applique à **toutes** les lignes de la table. On vérifie toujours deux fois.

Relier les tables : la vraie force

Le génie du modèle relationnel, c'est de **relier** les tables entre elles. Un client a

plusieurs commandes ? On met dans la table **commandes** une **clé étrangère** qui pointe vers l'**id** du client. Pour rassembler les informations dispersées, on fait une **jointure** (**JOIN**) : « donne-moi chaque client **avec** ses commandes ». C'est ce qui permet de modéliser le monde réel, où tout est connecté.

Dessiner avant de construire

On ne bâtit pas une maison sans plan, ni une base sans **modèle**. Avant la première ligne de SQL, on **schématise** : quelles tables, quelles colonnes, quelles relations, et avec quelles *cardinalités* (un client a-t-il une ou plusieurs commandes ?). C'est l'objet de la modélisation (avec **UML**, enseigné en cours). Un bon schéma évite des semaines de galère plus tard.

*À retenir : **SELECT** lit, **INSERT** crée, **UPDATE** modifie, **DELETE** supprime ; **WHERE** filtre. On relie les tables avec des clés étrangères et des jointures. Et on modélise la base avant de la construire.*

Ce qu'il faut retenir de ce chapitre

- Au-delà de quelques données, on passe du fichier à une vraie base de données.
- Le modèle relationnel range tout en tables reliées par des clés.
- SQL est le langage universel : CRUD + **WHERE**, et les jointures pour relier.
- On dessine (modélise) la base avant de la coder.

Au cahier de la **semaine 7**, tu montes un carnet de notes avec deux tables reliées et tu en extrais les moyennes par une jointure. Tu sais désormais stocker pour de vrai.

Chapitre 8 — Comment les machines se parlent

Une lettre qui voyage

Quand tu envoies un message d'un ordinateur à un autre — à l'autre bout du monde — comment fait-il pour arriver, intact, à la bonne machine ? C'est tout le sujet des **réseaux**, un pilier de ta formation (le cursus **Cisco CCNA** y est dédié).

L'analogie qui éclaire tout : envoyer une lettre. Tu écris le texte, tu le mets dans une enveloppe, tu inscribes l'adresse, la poste l'achemine de centre de tri en centre de tri, et le facteur la dépose. Un message informatique fait exactement ce voyage, en traversant des **couches** successives.

Le modèle OSI : sept étages

Pour organiser ce voyage, on a défini un modèle en **7 couches**, le modèle **OSI**. Chaque couche a un rôle précis et ne s'occupe que du sien : la plus haute, c'est l'application (ce que tu vois) ; la plus basse, c'est le signal physique (le câble, le Wi-Fi). Entre les deux, des couches gèrent l'adressage, le routage, la fiabilité.

Tu n'as pas besoin de tout retenir par cœur tout de suite, mais deux couches sont reines : la couche **réseau** (l'**IP**, qui gère les adresses) et la couche **transport** (**TCP** et **UDP**, qui gèrent la livraison). TCP, c'est le courrier recommandé (livraison garantie) ; UDP, c'est la carte postale (rapide, sans garantie).

À retenir : le modèle OSI décrit en 7 couches le voyage d'un message. Retiens surtout IP (les adresses) et TCP/UDP (la livraison fiable ou rapide).

L'adresse IP : l'adresse postale des machines

Chaque machine a une **adresse IP** : quatre nombres de 0 à 255 (en IPv4), comme **192.168.1.10**. C'est son adresse postale sur le réseau. Une adresse spéciale, **127.0.0.1** (« localhost »), désigne **toujours ta propre machine** — pratique pour tester.

L'adresse se divise en deux : une partie qui désigne le **réseau** (le quartier) et une partie qui désigne la **machine** (le numéro de maison). C'est le **masque de sous-réseau** qui indique où passe la frontière.

Le subnetting : découper intelligemment

Une grande entreprise ne met pas toutes ses machines dans un seul gros réseau : elle le **découpe** en **sous-réseaux** (un par service, par exemple), pour mieux organiser et sécuriser. C'est le *subnetting*, un grand classique des exercices Cisco. Et devine sur quoi ça repose ? Sur le **binaire** que tu as vu dès la semaine 1. Une

adresse IP n'est qu'une suite de bits ; découper, c'est jouer avec ces bits. Le Coin Maths du programme prend ici tout son sens.

Les services qui font tourner le réseau

Trois acteurs invisibles te facilitent la vie :

- Le **DHCP** te donne automatiquement une adresse IP quand tu te connectes (l'hôtesse qui t'attribue ta chambre).
- Le **DNS** traduit les noms ([google.com](https://www.google.com)) en adresses IP (l'annuaire téléphonique) — sans lui, il faudrait retenir des suites de chiffres.
- Les **ports** identifient un service précis sur une machine (l'appartement dans l'immeuble) : 80 pour le web, 443 pour le web sécurisé, 22 pour l'administration à distance.

À retenir : une adresse IP situe une machine ; le masque sépare réseau et machines ; DHCP attribue les IP, DNS traduit les noms, les ports désignent les services.

Ce qu'il faut retenir de ce chapitre

- Un message voyage en traversant des couches (modèle OSI, 7 couches) ; IP et TCP/UDP sont les plus importantes.
- L'adresse IP situe chaque machine ; **127.0.0.1** = soi-même.
- Le subnetting découpe un réseau en sous-réseaux, et repose sur le binaire.
- DHCP (attribue les IP), DNS (traduit les noms) et les ports (désignent les services) font tourner le tout.

Au cahier de la **semaine 8**, tu construis un vrai petit réseau dans un simulateur et tu conçois le plan d'adressage d'une entreprise. Tu commences à penser comme un administrateur réseau.

Chapitre 9 — Monter ses serveurs

Un ordinateur dans l'ordinateur

Les sites web, les applications, les services en ligne tournent sur des **serveurs** : des ordinateurs allumés en permanence, quelque part dans un centre de données. Mais acheter une machine physique par service serait ruineux et gâché (un serveur n'est utilisé qu'à une fraction de sa puissance). La solution s'appelle la **virtualisation** : faire tourner **plusieurs ordinateurs simulés** (des **machines virtuelles**, ou VM) sur une seule machine physique.

Le chef d'orchestre s'appelle l'**hyperviseur**. Tu en utilises déjà un pour apprendre (**VirtualBox**) ; en entreprise, on trouve **VMware ESXi** et **Proxmox**, enseignés en cours. C'est exactement la technologie qui fait fonctionner le **cloud** : quand tu loues un serveur chez un hébergeur, c'est presque toujours une VM.

Un superpouvoir des VM : le **snapshot**, une photo de l'état complet de la machine à un instant donné. Tu expérimentes, tu casses tout, et tu restaures la photo en quelques secondes. Un filet de sécurité magique.

À retenir : une VM est un ordinateur simulé ; l'hyperviseur les fait tourner ; le snapshot fige un état restaurable. C'est le fondement du cloud.

Administrer à distance, en ligne de commande

Un serveur n'a en général **ni écran ni clavier**. On l'administre **à distance**, par le réseau, via un canal sécurisé appelé **SSH**. Te voilà récompensé d'avoir appris le terminal en semaine 1 : c'est exactement comme ça qu'on pilote un serveur — en tapant des commandes, à distance.

Le quotidien de l'administrateur : installer des services (un serveur web, une base de données), vérifier qu'ils tournent, les redémarrer, gérer les **utilisateurs** et leurs droits, et lire les **logs** (les journaux dans `/var/log/`, qui racontent tout ce qui se passe). Côté Windows, le monde de l'entreprise s'organise autour de **Windows Server** et de l'**Active Directory**, l'annuaire central qui gère tous les comptes et tous les droits d'une organisation.

Fermer les portes : le durcissement

Un serveur fraîchement installé est une **maison aux portes grandes ouvertes**. Le **durcissement** (*hardening*) consiste à les fermer pour réduire la prise offerte aux attaquants. Les réflexes de base, qui suffisent déjà à écarter l'immense majorité des menaces :

- **mettre à jour** régulièrement (les failles connues sont les plus exploitées) ;
- activer un **pare-feu** et n'ouvrir que les ports strictement nécessaires ;

- sécuriser l'accès **SSH** (pas de connexion directe en administrateur, des clés plutôt que des mots de passe) ;
- appliquer le **moindre privilège** : chacun n'a que les droits dont il a besoin, pas plus.

À retenir : un serveur se sécurise dès l'installation. Mises à jour, pare-feu, SSH durci et moindre privilège : ces quatre réflexes éliminent déjà l'essentiel du risque.

Et si tu veux aller bien plus loin dans la défense — détecter les intrusions, analyser les attaques, répondre à un incident — c'est tout l'objet de l'autre programme, « **Alex le CyberHero** ». First Blood te donne les fondations système ; CyberHero t'apprend à les défendre.

Ce qu'il faut retenir de ce chapitre

- La virtualisation fait tourner plusieurs VM sur une machine ; l'hyperviseur les orchestre ; le snapshot sauve la mise.
- On administre un serveur à distance en ligne de commande (SSH) : services, utilisateurs, logs.
- Windows Server + Active Directory gèrent les comptes en entreprise.
- Le durcissement ferme les portes inutiles : mises à jour, pare-feu, SSH, moindre privilège.

Au cahier de la **semaine 9**, tu montes un vrai serveur web dans une VM, tu héberges ton propre site, et tu le durcis. Tu deviens administrateur.

Chapitre 10 — Tout réunir

La cryptographie : le coffre-fort du numérique

Comment envoyer un mot de passe sur Internet sans que quelqu'un puisse le lire au passage ? Comment un site stocke-t-il tes identifiants sans pouvoir les révéler, même s'il se fait pirater ? La réponse tient en un mot : la **cryptographie**.

Deux idées à ne surtout pas confondre :

Le chiffrement rend un message illisible, puis lisible à nouveau grâce à une **clé**. C'est **réversible**. Il y a deux saveurs : le chiffrement **symétrique** (la même clé pour fermer et ouvrir, comme une serrure classique — rapide, ex. AES) et le chiffrement **asymétrique** (une clé publique pour fermer, une clé privée pour ouvrir — c'est ce qui sécurise tout le web via HTTPS, ex. RSA).

Le hachage, lui, est **irréversible** : il transforme une donnée en une **empreinte** unique de taille fixe, impossible à inverser. C'est ce que fait le petit script `verifie.sh` que tu utilises depuis la semaine 1 ! On stocke les mots de passe **hachés** : même volé, un hash ne révèle pas le mot de passe d'origine. On ne le « déchiffre » jamais — on recompare des empreintes.

À retenir : chiffrer, c'est cacher puis révéler (réversible, avec une clé). Hacher, c'est transformer en empreinte unique et définitive (irréversible). Les deux protègent tes données, mais pas de la même façon.

Le droit : coder dans les règles

Un professionnel de l'informatique ne manipule pas les données n'importe comment. Le **RGPD**, la loi européenne, encadre l'usage des **données personnelles** (tout ce qui identifie une personne). Ses principes : demander le **consentement**, ne collecter que le **nécessaire**, dire **pourquoi**, respecter les **droits** des personnes (consulter, corriger, effacer leurs données), et les **protéger**.

Ce n'est pas un détail : les amendes se comptent en millions. Et cela rejoint la règle d'or de tout ce programme — celle de l'**article 323-1** du Code pénal : on n'accède jamais à un système sans autorisation. Le bon professionnel connaît la technique **et** la ligne jaune.

L'automatisation et l'IA : la boîte à outils moderne

Ton programme mentionne aussi des outils d'**automatisation** (comme **N8N**, qui relie des services entre eux par blocs, sans coder) et la **gestion d'agents IA**. Tu n'as pas à en devenir expert maintenant : retiens simplement que l'IA et l'automatisation font désormais partie du quotidien du développeur et de l'analyste en sécurité. Savoir les faire travailler pour toi est un atout.

Le projet final : la preuve par l'exemple

Tout ce que tu as appris ne vaut que si tu sais l'**assembler**. C'est le sens du projet **fullstack** final : une application web complète qui réunit l'interface (semaine 5), la logique serveur (semaine 6), la base de données (semaine 7), une connexion sécurisée avec mots de passe **hachés** (semaines 6 et 10), le tout versionné avec **Git** (semaine 1) et déployable sur ton **serveur** (semaine 9).

Une application comme celle-là, propre et publiée sur ton GitHub, c'est la **meilleure preuve** que tu es prêt — bien plus parlante qu'une ligne sur un CV.

À retenir : « fullstack » veut dire toute la pile, du visible au caché. Ton projet final prouve que tu sais faire dialoguer interface, serveur et base de données — en sécurisant les données.

Le mot de la fin

Alexandre, arrivé ici, tu as parcouru un chemin énorme : le terminal, deux langages de programmation, le web de bout en bout, les bases de données, les réseaux, les serveurs, la sécurité des données. Tu n'arrives pas en Bachelor en spectateur : tu arrives **avec une avance considérable**, et l'habitude de **pratiquer**.

Le « premier sang » est versé. Le vrai combat — tes études, tes projets, ta carrière — commence maintenant. Et tu es prêt.

Ce qu'il faut retenir de ce chapitre

- Chiffrement (réversible, avec clé : symétrique/asymétrique) $\neq$ hachage (irréversible, empreinte).
- Le RGPD encadre les données personnelles ; le cadre légal fait partie du métier.
- Automatisation et IA sont des outils modernes à connaître.
- Le projet fullstack réunit toutes les briques du programme : c'est ta preuve d'être prêt.

Au cahier de la **semaine 10**, tu construis ton application finale sécurisée. Le dernier badge t'attend. Va le chercher.